OpenZeppelin | security

OpenZeppelin Compact Contracts Audit



August 18, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 4

Scope 5

System Overview 6

Security Model and Trust Assumptions 7

Privileged Roles 9

Additional Considerations 9

High Severity 11

H-01 Account Identifiers And Encryption Secrets Share Undomain-Separated Hash 11

H-02 Memo Clearing Is Unconditional and Cannot Exclude Unread Credits 12

H-03 Missing Contract Commitment Claim When Recipient is a Contract 13

Medium Severity 14

M-01 Escrow Owner Memo Encrypted Under Randomness Controlled by Spender 14

M-02 Missing Running Total Balance Check Can Lock Balances Permanently 15

M-03 Pruning and Sweeping Authenticate the Account Secret Without The Encryption Secret 16

M-04 _approve Permits Recording Approvals for Non-existent Tokens, Breaking Downstream Circuits 17

Low Severity 18

L-01 Credit Randomness Nonce Can Be Reset By The Recipient 18

L-02 Memo Growth Through Inefficient Allowance Revocation 19

L-03 clearMemos Does Not Return the Authenticated Identifier 20

L-04 Missing Recovery Path for Balances Whose Plaintext Can No Longer Be Stated 21

L-05 Zero-Value Transfers Can Be Used To Deny Service To Genuine Credits 22

L-06 Inefficient Module Composability Can Lead To Confused Deputy Behavior 23

L-07 Allowance Placeholder Writes a Zero Ciphertext Where an Encryption of Zero Is Required 24

Notes & Additional Information 25

N-01 Homomorphic Helpers Cannot Verify That a Ciphertext Was Encrypted Under the Supplied Key 25

N-02 Clarify Documentation Around Deterministic Encryption Of Zero And Resulting Metadata Leakage 25

N-03 ConfidentialFungibleToken Allowance Semantics Diverge From ERC-20 And Are Undocumented 26

N-04 Curve Point Representation is Relied On by a Security Guard and Persisted State but Is Never Specified 27

N-05 Incomplete Documentation For Native Token Supply Wrappers 28

N-06 Lack Of Two-Step Ownership Transfer Modules 28

N-07 Missing Relation Between ownPublicKey and Its Associated Private Key 29

N-08 Owner Commitments Are Not Truly Distinct Over Multiple Contracts 30

N-09 Repeated Grants Can Exhaust Role Capacity And Permanently Block New Grants 30

N-10 Coin Information Is Recoverable Only From Discardable Return Values 31
N-11 Unbounded Per-Domain Supply Keys Can Create Persistent, Irreclaimable State Entries 32
N-12 Role Renunciation Allows Unbounded Nullifier Growth 33

Conclusion 34

Appendix 35
Issue Classification 35

Summary

Type	Privacy
Timeline	From 2026-08-03 To 2026-08-14
Languages	Compact

Total Issues	26 (0 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	3 (0 resolved)
Medium Severity Issues	4 (0 resolved)
Low Severity Issues	7 (0 resolved)
Notes & Additional Information	12 (0 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a diff audit of the OpenZeppelin/compact-contracts repository at release/v0.3.0-alpha.1-rc.0 branch with base commit at 9b6b21a and target commit at 0e9d659.

In scope were the following files:

```
contracts
└── src
    ├── access
    │   ├── AccessControl.compact
    │   ├── Ownable.compact
    │   ├── ShieldedAccessControl.compact
    │   └── ZOwnablePK.compact
    ├── crypto
    │   ├── EcdhMask.compact
    │   └── ElGamal.compact
    ├── security
    │   ├── Allowlist.compact
    │   ├── Blocklist.compact
    │   ├── Initializable.compact
    │   └── Pausable.compact
    ├── token
    │   ├── extensions
    │   │   ├── ConfidentialFungibleTokenPublicSupply.compact
    │   │   ├── NativeShieldedTokenDerivedNonce.compact
    │   │   ├── NativeShieldedTokenFamilyPublicSupply.compact
    │   │   ├── NativeShieldedTokenPublicSupply.compact
    │   │   └── NativeShieldedTokenPublicSupplyCore.compact
    │   ├── ConfidentialFungibleToken.compact
    │   ├── FungibleToken.compact
    │   ├── MultiToken.compact
    │   ├── NativeShieldedToken.compact
    │   ├── NativeShieldedTokenCore.compact
    │   ├── NativeShieldedTokenFamily.compact
    │   └── NonFungibleToken.compact
    └── utils
        └── Utils.compact
```

System Overview

The Compact Contracts library provides reusable modules for building contracts on Midnight, a network whose contracts execute as zero-knowledge circuits written in the Compact language. It covers fungible, non-fungible and multi-token standards, two confidential token models, access control, and supporting cryptographic and lifecycle utilities. Release 0.3.0-alpha.1 adds the confidential and native shielded token families together with the ElGamal and ECDH primitives they depend on. Thirteen modules introduced in this release were reviewed in full at the target commit, and ten pre-existing modules were reviewed for the changes made to them since `9b6b21a`.

Compact provides no inheritance, so composition works differently from Solidity libraries. A module declares circuits and ledger state, and a contract composes modules by importing each under a prefix and delegating to it explicitly. Modules are consequently not deployable on their own. The composing contract decides which circuits become entry points, supplies all access control, and calls each module's initializer from its constructor. Ledger state is allocated once per module rather than once per import path, so two modules that wrap a shared core also share that core's state. Import prefixes namespace circuits but not witness declarations, which occupy a single flat namespace in the composed contract.

Account-based tokens: `FungibleToken`, `NonFungibleToken`, `MultiToken` and `ConfidentialFungibleToken` hold balances in ledger maps keyed by an account identifier that each circuit derives internally from a caller-supplied secret. The new `ConfidentialFungibleToken` contract stores balances as exponential ElGamal ciphertexts, so a balance can be proven but not read directly, and recovering an amount from a ciphertext alone requires a discrete-logarithm search. Amounts therefore reach recipients through a separate channel, a per-account memo list encrypted under an ECDH one-time pad, and a wallet reconstructs its balance by folding those memos into a local cache. Incoming value arrives in a pending pool, which is moved into the spendable balance by whoever holds the account secret. Allowances are funded rather than promised: approving debits the owner's balance and places the amount in an escrow held as two ciphertexts, one readable by each party.

Native shielded tokens: the `NativeShieldedToken` family wraps Midnight's Zswap coins, so value exists as discrete coins rather than as balances. A coin carries a color derived from a domain separator and the deploying contract's own address, which is what restricts minting of

that color to that contract. Two flavors sit on a shared core: `NativeShieldedToken` issues a single token type, and `NativeShieldedTokenFamily` issues many from one contract, which the absence of contract-to-contract calls makes necessary. Both allocate the same core state, so only one may be composed per contract. Optional extensions add public supply tracking and derived mint nonces, the latter drawn from a public counter that makes the mint linkable where a secret nonce would not. These modules expose minting and burning only. Transfers of an already-delivered coin occur entirely at the protocol layer, with no contract call involved and no transfer circuit present, so a composing contract cannot observe or restrict them. Contract-initiated sends also produce no coin ciphertext, which means the coin information returned by a mint or a refund is the only copy in existence.

Access control and lifecycle modules: `Ownable` and `AccessControl` identify callers by a public account identifier. `ZOwnablePK` and `ShieldedAccessControl` keep that identity private by storing a commitment and requiring a caller to reproduce it in-circuit, the latter recording grants as leaves of a fixed-capacity Merkle tree and revocations as nullifiers. An append-only Merkle tree supports membership proofs but not their negation, so a contract cannot establish that a leaf is absent. Deduplicating grants is therefore not expressible in-circuit, and the capacity of the tree bounds the number of grants a deployment can ever record. `Pausable`, `Initializable`, `Allowlist` and `Blocklist` supply mechanisms without policy: each exposes a check and leaves the decision of when that check applies to the composing contract. Membership in `Allowlist` and `Blocklist` is publicly readable and the approved or blocked set is publicly enumerable, so composing either with a confidential asset republishes the set of eligible accounts and bounds the anonymity set, which the modules state and for which they point to a shielded variant as the alternative.

Security Model and Trust Assumptions

Contract-to-Contract calls: the compact language does not currently support contract-to-contract (C2C) calls. This has a direct consequence for access control: a contract cannot pass any authentication check that is evaluated by another contract. The library acknowledges this limitation through an explicit naming convention. As a safety precaution, the standard functions will refuse to assign ownership, tokens and roles to contract addresses. However, there are functions prefixed with `_unsafe` (such as `_unsafeGrantRole` in `AccessControl`) that developers can use to bypass this restriction.

Composing contract: the composing contract is the primary trust boundary in this library rather than a peripheral concern. It determines which module circuits become callable, so exposing an internal circuit is equivalent to publishing an unrestricted entry point, and it supplies every access-control check, since no module gates its own operations. Each module documents the obligations that composing it correctly entails, covering initialization, supply accounting, which circuits must never be re-exported, which return values must be retained, and where naming conventions must be observed. Integrators should assume that a deployment's security properties are determined mainly by this layer, and that a review of the library alone does not establish them. The conventions that govern these obligations are documented by the repository.

Contract Maintenance Authority: a deployed contract's circuits are not immutable. Midnight provides a per-contract maintenance authority, a signing committee rather than a single key, which can rotate the verifier keys behind a contract's entry points and replace itself, but cannot migrate ledger state, as the library itself records. Any property established in this report therefore holds for the circuits deployed at the time of review and for as long as that committee is trusted, and the mechanism itself was not reviewed as part of this engagement.

Wallet and witness provider: this actor is not an on-chain role and is the most consequential assumption in the model. Witnesses are offchain code supplied by the prover, are not verified by the circuit, and are not constrained to return the same value on two calls within one circuit. The wallet is assumed to supply a fresh, high-entropy and secret randomness seed on every invocation, since confidentiality on every value-bearing path depends on it and a repeated seed reveals differences between amounts. It is assumed to derive an account secret and an encryption secret from independent material, because the two derivations are not domain-separated from each other. It is assumed to supply a unique nonce for each mint, to retain the plaintext balance a spend must state, and to preserve any coin information returned to it.

Midnight platform: the compiler, runtime and ledger are treated as an external dependency, and several properties are relied upon rather than enforced in circuit. The runtime is assumed to constrain every curve point into the Jubjub prime-order subgroup used in `ElGamal` and `EcdhMask`, and these cryptographic modules perform no in-circuit subgroup check on that basis. Transaction validity is assumed to turn on the values a transcript recorded reading, so a transaction remains valid when those values are unchanged even if other parts of the same state were modified. Atomicity is assumed to be delimited by checkpoints rather than by circuit boundaries, and these modules contain none, so a failed assertion leaves no partial state behind.

The byte-to-field conversion: used to derive scalars in cryptography contracts - `ElGamal` and `EcdhMask` is assumed to truncate such that every derived scalar is a valid Jubjub scalar,

a property the modules state but which is not documented by the platform. The security of those derivations is monotone in that output width, and the failure is asymmetric: a wider image would push most derived scalars past the subgroup order and fault loudly, whereas a narrower one would reduce key strength while every test continued to pass. Because those scalars occupy a subinterval below two to the power of 248 against a subgroup order slightly above two to the power of 252, a formal argument for the ElGamal construction rests on the short-exponent variant of the Diffie-Hellman assumption rather than the plain one, at an accounting cost of under one bit.

Replay protection: replay is assumed to be prevented above the contract, at the intent level. A duplicate intent hash is rejected, an expired intent is rejected, and a call repackaged into a fresh intent fails the binding commitment check because it is bound to its parent's binding

Counterparties: any registered account can credit any other in `ConfidentialFungibleToken` contract, which appends an entry to the recipient's memo list and raises their pending balance. A counterparty is therefore able to influence recipient state without permission.

Privileged Roles

The specific privileged roles vary by module, but the general trust model is consistent across the library: accounts holding administrative roles are assumed to be honest. The library provides no on-chain mechanism to limit, time-lock, or reverse the actions of a legitimately credentialed admin. Implementing contracts are responsible for establishing governance processes that reduce the risk of admin key compromise or misuse.

Additional Considerations

- The multisig and ECDSA modules are outside the scope of this engagement. Contract-to-contract interaction is out of scope for the confidential token, which documents that a contract cannot hold a confidential balance because it has no private state in which to custody an encryption key. Deployability is a property of the target network rather than of the library, since the per-transaction byte budget is a governance-configurable node parameter and a full-surface deployment was observed to exceed it on a development stack.

- Given the developing nature of the compact language, there can be possibility of undiscovered compiler based issues that may generate unintended behavior for the circuits.

High Severity

H-01 Account Identifiers And Encryption Secrets Share Undomain-Separated Hash

The account identifier derivation in `Utils` hashes a secret key using `persistentHash` over a single-element vector of `Bytes<32>`. The derivation of the ElGamal secret scalar applies the identical construction to its input and then truncates the result into the Jubjub scalar field. Neither includes a domain separation tag, so both compute the same function of the same input.

The first output is public. `ConfidentialFungibleToken` stores it as the map key for balances, encryption keys, memo lists and escrow entries, all of which are readable on chain. The second output is the private key protecting every ciphertext held under that identifier. For any 32-byte value used in both roles, the private scalar is a truncation of a value published on chain.

The two secrets arrive through separate witnesses, `wit_ConfidentialTokenSK` and `wit_ConfidentialTokenEK`, and no circuit constrains them to differ. The condition for exposure is precise: the 32-byte value used as the encryption secret must never also be a value hashed into an account identifier. A wallet returning the same value from both witnesses satisfies that condition directly, and because the account identifier helper is shared across the access control and token modules, so does a wallet that reuses one secret as an identity in any of those modules and as the encryption secret here. In either case an observer reads the identifier from the ledger, applies the same truncation the circuit applies, and holds the account's encryption secret, which opens the spendable balance, the pending balance, both ciphertexts of every allowance the account granted or received, and every entry in its memo list, requiring no privileged access and no computation beyond one hash truncation.

Consider adding distinct domain separation tags to both derivations, so that a shared input produces unrelated outputs and reuse of a single secret across the two roles becomes harmless. Consider also documenting whether integrating wallets are permitted to derive the account secret and the encryption secret from common key material, and if so, stating that domain separation is what makes the practice safe.

H-02 Memo Clearing Is Unconditional and Cannot Exclude Unread Credits

Incoming credit is delivered through a per-account memo list, and the the channel a recipient's wallet scans and fold into its local plaintext cache. The need is structural, because the stored balance is exponential ElGamal and yields only a curve point on decryption, so the memo carries the one representation of the amount that the recipient can actually state. Each credit performs both halves of that delivery, raising the pending ciphertext and appending the memo. Pruning is offered through `clearMemos`, which replaces the caller's entire list with an empty one and takes no argument identifying which entries the caller has read.

The two halves of a credit can therefore be separated by an intervening prune:

1. A wallet reads its memo list and folds every entry into its durable cache locally.
2. The wallet submits `clearMemos`.
3. A credit to the same account is ordered first, raising the pending ciphertext and appending a memo the wallet has never seen.
4. `clearMemos` executes and replaces the list, discarding that memo.
5. The pending increase remains on chain with no surviving record of the amount it represents.

The circuit already warns that clearing is destructive and tells callers to clear only memos they have already folded into a durable backup, but that instruction cannot be followed. Nothing in the call expresses which memos the caller intends to remove, so there is no expected count, no epoch and no list digest against which the circuit could reject a list that grew after the wallet last looked. A caller who does everything the warning asks still loses any memo appended between reading the list and the transaction executing, and the appending party is a third party sending value rather than the caller. This is an interaction between two independently valid transactions rather than a failed assertion inside one, so the guarantee that a reverting circuit leaves no state behind does not apply. That both remain valid follows from what each one reads. A credit reads the list contents, taking its length as a randomness nonce, so two credits to one recipient genuinely conflict, whereas `clearMemos` reads only whether the account has a list at all and then overwrites it with a constant, and appending an entry leaves that membership unchanged. Its recorded reads therefore still hold at inclusion even though the write discards what arrived in the meantime. Confirmation that a transaction recording only a membership check remains valid after another transaction changes that key's value would settle the behavior described here.

The credited amount is then stranded, because the recipient cannot name it and every spending path requires them to. Sweeping does not help and makes matters worse, since

`sweep` folds pending into spendable with no plaintext claim, so it succeeds and merges an unknown quantity into the balance that debits draw from. From that point the claim a debit demands can no longer be produced for any value, and the account's whole balance becomes unspendable rather than only the lost credit. A recipient can still search for a small missing amount by discrete logarithm against the point they can decrypt, so the outcome is recoverable for dust, but the per-operation ceiling is now the full unsigned 128 bit range, so a sender choosing a large amount places the value beyond any search, whether by accident or deliberately against a prune the sender can observe.

Consider making pruning state aware by having `clearMemos` accept the caller's view of the list, such as an expected entry count or a per-account epoch that increments on every append, and rejecting the call when it does not match current state, so that a list which grew since the caller last read it is never silently discarded. Consider also making pruning non destructive by removing a caller-specified prefix rather than replacing the list, so that entries arriving during the call survive it.

H-03 Missing Contract Commitment Claim When Recipient is a Contract

The _mint circuit and _burn circuit in the `NativeShieldedTokenCore` contract mint tokens or refund tokens via `mintShieldedToken` / `sendImmediateShielded` to a caller-supplied recipient (mint recipient or refund address), which is typed as `Either<ZswapCoinPublicKey, ContractAddress>`. When `refundTo` is a plain public key, this is a valid, self-contained Zswap output.

However, when `refundTo` resolves to `right<...>(ContractAddress)`, the resulting coin commitment is contract-associated and, per the ledger's well-formedness rules, must be claimed as a shielded receive by that contract in the same transaction/segment via `kernel.claimContractCall` - otherwise the transaction is rejected.

Consider accepting the target contract's entry point and call-randomness as additional witness/public inputs, computing `transientCommit`, and calling `kernel.claimContractCall` as per this example. Alternatively, consider only accepting `ZswapCoinPublicKey` (via `left<...>`) as the recipient if contract-address refunds are not a supported use case, to prevent constructing transactions that will always fail.

Medium Severity

M-01 Escrow Owner Memo Encrypted Under Randomness Controlled by Spender

An allowance is recorded as an escrow entry holding two ciphertexts and an owner-readable memo. Both ciphertexts encode the amount in the exponent, so neither party can cheaply recover the number, and the memo is therefore the owner's only channel for learning the live remaining allowance. That memo is an ECDH one-time pad addressed to the owner's public key. Because the recipient key is public ledger state, the ephemeral is the only secret standing between an observer and the plaintext, and the `EcdhMask` module states this requirement and assigns it: the ephemeral must be secret and unpredictable rather than merely distinct, and the caller supplies it and owns both properties.

In the escrow spend path that assignment breaks. The ephemeral is expanded from a seed supplied by the caller, but the caller at that moment is the **spender** rather than the **owner**. The expansion is a deterministic public hash, so possession of the seed is possession of the ephemeral, and the value protected belongs to a party absent from the call who contributes no entropy to it.

Two routes reach the plaintext: - A spender who publishes the seed allows any observer to recompute the mask and read the owner's remaining allowance directly from public state. - A spender who reuses one seed across two spends against the same owner repeats the pad, so the difference of the two stored memos equals the difference of the two plaintexts and requires no secret at all.

The second route requires no intent, leaves the spender unaware, and is the behavior of the only witness present in the repository. The same construction in `approve` is sound precisely because the caller there is the owner.

The design already models the spender as hostile for the integrity of this field, since the remaining amount is recomputed in-circuit so that the spender cannot forge it. Its confidentiality is nevertheless left to that same spender's discretion, and the owner has no remedy. An owner who rotates a fresh high-entropy seed on every call they make still leaks, because the leaking call is not theirs, and nothing in escrow state records that a pad repeated. The authority conferred by `approve` extends to spending up to the cap and not to disclosing it, and the disclosure survives revocation.

Consider folding a per-escrow monotonic counter into the memo ephemeral. The addition worth noting is that the `_credit` path already applies exactly this mechanism, folding the recipient's memo-list length into its randomness so that each credit remains unique even when the seed repeats, and recording in its comment that the freshness is essential for the memo one-time pad. The remedy is therefore not a new construction but the one used a single function away. Applying it removes the accidental repetition that carries most of the practical exposure, though no construction prevents a spender who deliberately publishes the ephemeral, since any construction must let the spender compute the pad. It is further advisable to state which party each confidentiality property protects, so that a wallet author supplying the seed understands that the hygiene guards a counterparty rather than themselves.

M-02 Missing Running Total Balance Check Can Lock Balances Permanently

Balances are held as ElGamal ciphertexts and grow by homomorphic addition, incoming value being folded into the pending pool by the credit path and later merged into the spendable balance by `sweep`. The only ceiling applied anywhere is a per-operation check on each individual value, which is the full unsigned 128-bit maximum. Both growth operations reduce to component-wise curve addition, the credit path reaching it through a helper that adds a fresh encryption of the incoming amount and `sweep` calling it directly on the two stored ciphertexts. Because the amount sits in the exponent, that addition accumulates in the scalar field of the subgroup rather than in the argument type, so a stored ciphertext can come to represent a quantity above the unsigned 128-bit range while remaining perfectly well formed.

No running total is held anywhere for a check to apply to. The accumulated quantity exists only as the exponent of a curve point, which no participant in a credit can read, since a sender holds neither the recipient's encryption secret nor their current plaintext. The quantity that would need bounding is therefore not available to be bounded at the point where it grows.

1. Credits accumulate into one account, each within the per-operation bound but summing past the unsigned 128-bit range.
2. The stored ciphertext remains valid and continues to decrypt, since the sum is taken in the scalar field of the subgroup, which is far larger.
3. The holder attempts a debit, an approval, or an escrow spend, each of which requires them to state the current plaintext and have it checked against the stored ciphertext.
4. No value of the witness type decrypts to the stored quantity, so the assertion cannot be satisfied by any claim and every one of those paths fails.

The balance stays intact on chain with no operation able to move it, affecting the debit path, `approve` and the escrow spend alike. The restriction is enforced in the constraint system and not merely by the compiler, the emitted representation carrying an explicit 128 bit range constraint on that witness, so a prover willing to submit an out of range claim is refused as well. No privileged role is needed to arrive there, since ordinary transfers between users accumulate into one recipient's pending pool, and the amounts required are large rather than unusual in kind.

Consider treating a bounded total supply as the invariant to enforce, either by requiring a supply layer whose existing overflow check on the tracked total is made mandatory rather than optional, with the trade-off of mint and burn amounts being publicly determined through total supply delta, or by lowering the per-operation ceiling far enough that no plausible sequence of credits reaches the scalar field, and by documenting that a deployment without a bounded total can reach an unrecoverable balance. Consider carefully before instead range checking the plaintext as pending is merged into spendable, because the pending pool is written by third parties: a sender able to push a recipient past the ceiling would then make that recipient's `sweep` unsatisfiable and block their incoming value indefinitely, which relocates the failure onto the party being paid and defeats the purpose of separating pending from spendable. The check cannot be moved earlier into the credit path either, since a sender has no way to learn the recipient's current plaintext.

M-03 Pruning and Sweeping Authenticate the Account Secret Without The Encryption Secret

An attacker who obtains only an account's identity secret, and not its encryption secret, can leave that account permanently unable to spend. The module keeps the two secrets separate by design, the first fixing which account a caller is and the second protecting what that account can read, and every circuit that moves value proves both, as proving the balance against the registered encryption key. Two exported circuits prove only the first. Pruning replaces the caller's memo list after deriving the identifier and nothing else, and `sweep` folds pending value into the spendable balance using homomorphic addition, which needs no plaintext and therefore proves no knowledge of the key, its registration check confirming only that a key exists.

Holding the identity secret alone is enough to call both, and calling both in order freezes the account:

1. A third party credits the account. The amount lands in the pending pool and a memo carrying it is appended to the account's list, which is the only channel through which the holder learns that figure, the stored balance itself decrypting only to a curve point.
2. The attacker prunes before the wallet has read that memo. The credited amount is now unknown to the holder.
3. The attacker sweeps. The unknown amount is added into the spendable balance.
4. The holder attempts any debit or approval. Each requires stating the plaintext of the spendable balance, which is now the previous balance plus an amount they cannot determine, so no claim satisfies the assertion and nothing can be moved.

Either step alone is survivable. Pruning by itself strands the credit in the pending pool while leaving the spendable balance intact and usable, and sweeping by itself is the ordinary operation the holder performs anyway. It is the pair that converts one unreadable credit into an account that cannot transact, and both halves are reachable with the same single secret. A small missing amount can still be recovered by searching for a discrete logarithm, but the per-operation ceiling is the full unsigned 128 bit range, so a large credit places it beyond any search.

The exposure assumes the two secrets hold different values. A wallet is permitted to derive both from the same material, and where it does an attacker holding one holds the other and these checks would change nothing. That is an argument for enforcing the separation wherever it is relied upon rather than against the separation itself.

Consider requiring an encryption secret witness in both circuits, asserted to re-derive the public key registered for the account, before either the memo list is replaced or pending value is merged, so that authority over what an account can read matches authority over what it can spend. Adding the check to only one of the two leaves the sequence above intact. Consider also making pruning remove an explicitly identified prefix rather than the whole list, which bounds what a caller holding one secret can destroy.

M-04 `_approve` Permits Recording Approvals for Non-existent Tokens, Breaking Downstream Circuits

The exported `_approve` circuit of `NonFungibleToken` performs its approver validation only when the `auth` argument is non-zero: the token-existence check sits inside the guarded

branch, while the approval write executes unconditionally. The zero-`auth` path is documented as a deliberate escape hatch, and its requirements list no condition that `tokenId` exist.

The mock contract `MockNonFungibleToken` exports the `_approve` circuit, bypassing the generation of `auth` through `computeAccountId` as implemented in approve circuit.

Consequently, following the mock contract implementation, an approval can be recorded for a token that does not exist, and the invariant that a populated approval implies an existing token no longer holds. That invariant is exactly what the module relies upon elsewhere: a comment in `_unsafeTransferFrom` states that supplying an `auth` argument verifies token existence and that the returned previous owner therefore need not be checked against zero. Two exploits follow. In the first, an approval is planted on an unminted identifier and the approvee calls the public `transferFrom` or `_unsafeTransferFrom` with the zero account as the source: the update circuit reads the owner as zero, passes authorization on the stale approval, skips the balance decrement, credits the recipient and writes the owner entry, **minting the token without mint authorization** and permanently blocking the composer's own gated mint of that identifier. In the second, an approval is planted on an identifier about to be minted legitimately; because the update circuit clears approvals only when the source is non-zero, a mint never clears the approval, so the planted approval survives and the planter transfers the freshly minted token away from its owner.

Consider enforcing token existence independently of approval state. For example, make _checkAuthorized revert for owner == Utils_zeroAccount() unconditionally (before evaluating approvals), and explicitly reject fromAddress == Utils_zeroAccount() in transferFrom / _unsafeTransferFrom. In addition, consider updating the mock contracts by removing all internal circuits that are not supposed to be exported to avoid integration issues for composers extending the contract.

Low Severity

L-01 Credit Randomness Nonce Can Be Reset By The Recipient

The credit path derives both its balance randomness and its memo ephemeral by hashing the recipient account together with the memo-list length. The comment beside it explains that the length serves as a monotonic nonce so that each credit differs even when the wallet reuses its seed. This protection matters because the memo is an additive one-time pad: a repeated

ephemeral to one recipient makes the difference of two stored memo values equal the difference of the two delivered amounts exactly, readable by anyone.

However, the length is not monotonic. Pruning replaces the recipient's list with an empty one, after which the counter walks the same sequence a second time and each credit following the prune pairs with the credit that held the same count before it. Every input to the derivation must match for a pair to form, the seed among them, so credits from different senders never pair with one another and the party that gains is a passive observer rather than either participant. What such an observer obtains is a relation between amounts rather than an amount, though two equal amounts produce byte-identical entries and a single amount learned outside the system converts every connected difference into an exact figure.

The module header records the reset and calls it an in-epoch backstop rather than a substitute for seed freshness. What it does not record is who ends an epoch. Pruning is available only to the account holder, so the counter guarding a sender's pad is advanced and reset by the recipient of that sender's transfers. The exposure still requires the sender to have reused a seed, so this describes the conditions under which a defence-in-depth measure stops functioning rather than a failure reachable against a correctly implemented wallet.

The addition worth noting is that the counter feeds both derived values and not only the memo ephemeral, so a revisited count repeats the balance randomness as well. Two such credits produce pending deltas that share an identical first component, which makes the reuse detectable without any secret, and whose second components differ by the amount difference expressed as a curve point.

Consider deriving the nonce from a per-account counter that only ever increases, stored independently of the memo list so that pruning cannot return it to an earlier value, which keeps the measure effective for exactly the wallets it exists to protect and removes a counterparty's influence over it. Moreover, consider describing in the seed guidance that the current counter is advanced by the recipient rather than by the sender whose randomness it protects.

L-02 Memo Growth Through Inefficient Allowance Revocation

Every credit appends an entry to the recipient's memo list, and the module records this as accepted residual griefing, stating that an attacker can spam dust credits to grow that list without bound, that only the recipient can clear it, that rejecting zero valued transfers would not help because dust at value one defeats it, and that a real fix is architectural and deferred. The behaviour is therefore known and accepted, and the reachability is as described, since the per-

operation check bounds a value from above only and a debit of zero satisfies its balance comparison, so any registered account can call transfer repeatedly at zero or one and append a memo to the target's list each time. Two aspects sit outside what that note covers.

The first concerns the remedy. The disclosure counts the victim's costs as storage, scanning and the gas to clear, which presents pruning as an expense. Pruning is more than an expense, because the circuit that performs it warns that clearing is destructive and can lock funds, the memo list being the only record of amounts that the balance ciphertext does not yield directly. An attacker who inflates a memo list is therefore not only imposing a cost but pressing the victim toward an operation the module itself describes as dangerous, and the more effective the spam the stronger that pressure becomes.

The second concerns revocation. Setting an allowance to zero is the conventional way to withdraw a spender's authority, but `approve` refunds and removes any prior escrow and then writes a fresh entry encrypting the new value, so a zero valued approval leaves an entry in place rather than none. The requires only that an entry exist with a sufficient recorded amount, a condition zero meets against zero, so the spender retains a working route into the credit path and can continue appending memos with zero value to any target after the owner believes their authority has ended.

Consider having a zero valued approval remove the escrow entry rather than store an empty one, so that withdrawing authority leaves no callable path behind. Consider also recording alongside the griefing note that the recipient's only available response is destructive and can render funds unspendable, so that the cost of the accepted limitation is stated in full while the architectural fix remains outstanding.

L-03 `clearMemos` Does Not Return the Authenticated Identifier

The module makes caller-side gating sound by having each caller-authenticating circuit return the identifier it authenticated. The header enumerates them as `register`, `_burn`, `transfer`, `_move`, `approve`, `transferFrom`, `sweep` and `_burnFrom`. `clearMemos` authenticates a caller in exactly the same way, deriving the account from `wit_ConfidentialTokenSK`, but returns `[]` and is absent from that list.

The contrast with `sweep` is direct: both are maintenance circuits that authenticate the identity secret alone and act only on the caller's own state, yet `sweep` returns the identifier and `clearMemos` does not.

However, that return value is what lets a wrapper gate without a witness of its own. A wrapper layering an allowlist or blocklist over the module needs the identifier the module actually authenticated, because as the header states, "*a separate witness call in the wrapper could return a different secret than the one this module authenticates with, letting a frozen account slip a clean id past the gate*". A witness is offchain code that may answer differently on two calls within one circuit, so any identifier a wrapper derives itself is unconstrained against the one the module used. The returned value is therefore the only sound source, and `clearMemos` supplies none. Furthermmore, allowing `clearMemos` to bypass allowlist/blocklist integrations is not ideal given its destructive behavior. Assume an edgecase scenario where a secret key is leaked and that account is blocklisted, the accessible `clearMemos` circuit can provide a window to the attacker to clear the memos before the owner/wallet sees and thus, permanently blocking the funds.

Consider returning the authenticated account identifier from `clearMemos` as the other caller-authenticating circuits do, and adding it to the header's enumeration so the list matches the code, which restores the guarantee for the one entry point currently outside it.

L-04 Missing Recovery Path for Balances Whose Plaintext Can No Longer Be Stated

Every path that removes value requires the caller to state the plaintext of their entire balance, which `ElGamal_assertDecryptsTo` binds to the stored ciphertext against a witness typed `Uint<128>`. Several independent conditions can leave an account holding value for which no such claim exists, including a credit whose memo was discarded before the wallet folded it into its cache, and a ciphertext that has accumulated past the range the witness type can express. The underlying mismatch is that homomorphic addition accumulates in the Jubjub scalar field, whose order is far above the `Uint<128>` maximum, so a well-formed ciphertext may encode a quantity the witness cannot name.

Once an account reaches that state, the condition is terminal. The value remains on the ledger and is provably present, yet every debit, approval, and escrow spend fails the assertion for every value the caller could supply. Remediations for the conditions that lead there are preventative in nature, so an account already affected has no exit, and no circuit in the module accepts a claim outside `Uint<128>` or otherwise re-expresses a balance the caller cannot state.

Consider providing a narrow recovery circuit that accepts a wider plaintext claim, binds it with the existing decryption assertion, and permits the holder to split the balance into pieces each within the ordinary range. Confining the wider claim to that circuit would leave the normal paths

and the module's stated `Uint<128>` balance range unchanged, while giving an affected account a route back to a spendable state.

L-05 Zero-Value Transfers Can Be Used To Deny Service To Genuine Credits

Delivering value reads the recipient's memo list, takes its current length as a nonce, folds that nonce into the randomness for the balance update and for the memo, rewrites that recipient's pending ciphertext, and appends to the same list. The length a credit reads is therefore part of what its transaction records having read, and appending the memo changes that same length. Every credit to one account therefore depends on a value it goes on to alter. Two credits prepared against the same view of this value cannot both remain valid, and the module records this as a documented limitation, acceptable for its first target use case of tokenized deposits.

The note describes a concurrency property where two ordinary payments arrive together and one needs a retry. The same mechanism is also available deliberately and at almost no cost. Reaching the credit path requires only a transfer, the per-operation check bounds a value from above and not from below, and a debit compares the caller's balance against the amount requested, a comparison that a zero amount satisfies for any registered account including one that holds the encrypted zero written at registration. A party wanting to disrupt a particular account can therefore send it repeated zero-valued transfers, each invalidating whatever genuine credit was prepared against the previous state, for the price of fees and no capital at all. Any credit has the same effect, since the conflict follows from the read and the append rather than from the amount, but a non-zero transfer pays value to the target and is limited by the sender's balance, whereas the zero-valued form is free and unbounded. The recipient has no response available, since the contention is on the state every incoming credit must touch, and clearing the memo list neither prevents nor slows it.

The use case named as making the limitation acceptable is also the one in which it is most impactful. A tokenized deposit address is published so that others may pay into it, is reused, and is expected to remain reachable, which makes it both an easy target and a costly one to have blocked. The disclosure and the exposure therefore point in opposite directions, and the module does not say that the conflict is reachable on demand by anyone rather than only by coincidence.

Consider giving each incoming credit its own key, so that concurrent payments to one recipient write to distinct locations and are combined when the recipient next sweeps, which removes the shared state that forces them to serialize. Consider at least rejecting zero-valued transfers, which does not address contention between genuine payments but does remove the free and

unlimited form of it, and stating in the concurrency note that the conflict can be produced deliberately, so that a deployment relying on inbound liveness evaluates it as an availability property rather than as an occasional retry.

L-06 Inefficient Module Composability Can Lead To Confused Deputy Behavior

Several modules derive the calling identity from their own witness. `Ownable` derives it from `wit_OwnableSK`, `AccessControl` from `wit_AccessControlSK`, and the account-based token modules each from their own secret, so that `FungibleToken` acts on `left(_computeAccountId())` derived from `wit_FungibleTokenSK`. Witnesses are offchain code supplied by the prover, and nothing constrains two of them to return the same value.

A composed circuit that authorizes through one module and acts through another therefore binds its authorization and its effect to different identities. A prover can satisfy the ownership witness with the real owner's secret so that `assertOnlyOwner` passes, satisfy the token witness with a secret of their own, and have the resulting transfer or mint credited to the second identity. The authorization check is sound in isolation and the effect is sound in isolation; only their pairing is unconstrained, which is the shape of a confused deputy and is not visible at the call site. The library already contains the remedy in one place: `ConfidentialFungibleToken` has each caller-authenticating circuit return the identifier it authenticated precisely so a wrapper gates the value that was actually used, and its header explains that a separate witness call in the wrapper could return a different secret. That convention was not propagated to the modules named above, whose internal identity derivations are non-exported, so a composer has no sound value to compare against.

Consider deriving one caller identity in the composing circuit and asserting equality across the modules it drives, or preferring the internal circuits which accept the principal as an explicit argument rather than reading a module-local witness. It is further advisable to state in each module that its derived identity is independent of every other module's, since a composed circuit that assumes a single caller is unsound unless that equality is added.

L-07 Allowance Placeholder Writes a Zero Ciphertext Where an Encryption of Zero Is Required

When no allowance exists the getter returns a synthetic entry in place of a zeroed default, and its documentation states that the entry is well formed, that its encrypted amounts decrypt to zero, and that it is safe for a wallet to decrypt. The entry carries three encrypted fields and the claim holds for two of them. The pair of ElGamal ciphertexts is filled with the identity point in both components, which under an encoding that places the amount in the exponent is a genuine encryption of zero and recovers zero on decryption. The third field is a memo, and it is built by a different scheme.

That scheme masks the value additively, so an encryption of zero under it is the mask itself rather than the number zero. The placeholder instead sets the memo's ciphertext to zero, which is a zero written into the slot an encrypted value occupies rather than an encrypted zero. Recovering a value multiplies the ephemeral public key by the recipient's secret and subtracts a hash of the resulting point, so what the reader obtains is the negation of that hash. The same fill with the neutral element was applied to both constructions and only one of them treats zero as neutral.

The recovered figure is identical for every reader, since the ephemeral is the identity point and multiplying it by any secret yields the identity, making the hash of that fixed point under the escrow domain tag a constant rather than a per-account value. It is also large, the mask being a truncation to thirty one bytes and the result its negation in the field, which places it just below the field modulus and far above the maximum a transfer amount may take. A wallet that range checks what it decrypts against the amount type rejects it at once, which is harmless, whereas one that narrows the field element to the width of an amount, an ordinary mistake when handling arbitrary precision integers, obtains a plausible figure and reports a substantial allowance for a relationship that was never created. Nothing becomes spendable, since the spending paths test escrow membership separately, so the consequence falls on what integrators display and record rather than on what can be moved. The same placeholder shape is built when a prior escrow is refunded, but that occurrence carries a note that the slot exists only so the refund reads an on-curve no-op and is removed before anything decrypts it, which leaves the getter as the one place the value reaches a reader.

Consider setting the memo's ciphertext to the hash of the identity point under the same domain tag, which is the encryption of zero for this scheme and makes the subtraction cancel, so that all three fields satisfy the claim the documentation makes for them. Consider alternatively returning a value documented as a sentinel for an absent allowance, with the

accompanying statement that wallets must recognise it rather than decrypt it, so that the getter's contract and its behaviour agree either way.

Notes & Additional Information

N-01 Homomorphic Helpers Cannot Verify That a Ciphertext Was Encrypted Under the Supplied Key

The ciphertext type carries only its two curve points and no data binding it to a recipient key. The helpers that fold a plaintext into an existing ciphertext or refresh its randomness accept the public key as a separate argument and have no means of checking it against the ciphertext they modify, so supplying a key other than the one the ciphertext was created under produces a result that decrypts to neither the original plaintext nor any recoverable value. Nothing in the circuit detects the substitution, and a consuming contract that persists the result stores state its rightful holder can no longer open or prove. Correctness is therefore an unstated precondition on every caller rather than a property the module enforces. The consumer in this release satisfies it, taking every key from a lookup in the registration map rather than from a parameter and validating each claimed plaintext against the stored ciphertext before a debit, with no circuit in that module accepting a caller-supplied key, so no path in the library as shipped reaches the failure.

Consider making the key association explicit at the interface, for example by storing a key identifier alongside each ciphertext and deriving the key internally rather than accepting it from the caller. Consider also documenting that these helpers must never receive a caller-controlled or mutable key where long-term decryptability matters.

N-02 Clarify Documentation Around Deterministic Encryption Of Zero And Resulting Metadata Leakage

The encryption of zero returns a fixed pair of identity points rather than a randomized ciphertext, and that value is persisted into the balance and pending maps at registration and

written back to pending on every sweep. An observer can therefore identify by inspection any slot still holding it, which discloses that an account is registered but never credited, or that a pending pool has just been swept. A zero arrived at by spending is not identifiable, because subtraction produces a randomized ciphertext, so what leaks is the provenance of the value rather than the value itself.

The deterministic form cannot be replaced because `_refundPriorEscrow` adds the stored owner copy onto the balance homomorphically, and only the identity leaves that ciphertext byte-identical on a first approve, so a randomized encryption of zero would re-randomize the owner's balance for no value moved. It would also extend the set of operations that depend on seed freshness and cost additional circuit rows, while offering no privacy gain, since `register` and `sweep` are public calls and the same facts are derivable from transaction history regardless. The remedy therefore lies in documentation rather than in the construction.

That property is recorded on `encryptZero` itself, which describes the value as not hiding and warns that a slot left at it is distinguishable as never written. It is absent, however, from the plaintext-disclosure enumeration in the `ConfidentialFungibleToken` header, which lists amounts, the supply delta, the approve cap and the counterparty graph. An integrator reasoning about what reaches the public ledger consults that enumeration rather than a primitive's docstring, and would otherwise carry an assumption of indistinguishability into a privacy interface.

Consider recording in the `ElGamal` module header, and as a further entry in the `ConfidentialFungibleToken` plaintext-disclosure enumeration, that a slot still holding the deterministic sentinel is publicly identifiable as never written or freshly reset, so that a single authoritative place exists for an integrator to consult.

N-03 `ConfidentialFungibleToken` Allowance Semantics Diverge From ERC-20 And Are Undocumented

The `approve` circuit debits the owner's spendable balance by `value` and funds the escrow with it, so an allowance is reserved rather than merely permitted and the sum of outstanding approvals is bounded by the owner's balance. This diverges from the ERC-20 semantics an integrator will assume, where allowances are unfunded and may exceed the balance across several spenders, and the module header's disclosure and design notes do not record the difference. A frontend ported from an ERC-20 token would therefore present `approve` as free and idempotent while the user's spendable balance silently drops.

The deduction in balance during approvals is in fact necessary due to the added confidentiality layer. At spend time, a spender would have to prove the owner's balance covers the allowance, which needs the owner's plaintext and therefore the owner's encryption secret, which the spender does not have.

Consider stating in the module header that allowances are funded at approve time, and that total outstanding approvals are therefore bounded by the owner's balance.

N-04 Curve Point Representation is Relied On by a Security Guard and Persisted State but Is Never Specified

Comparing a curve point against the identity is meaningful only when the representation those points take is specified, and this module leaves it unspecified while relying on such comparisons in two guards: ElGamal refusing an identity public key before encrypting and the hybrid module refusing both an identity recipient key and an ephemeral that reduces to the identity. The identity admits more than one spelling, since it can be obtained by multiplying the generator by zero or written directly as the coordinate pair with abscissa zero and ordinate one, and the module uses both. One production site takes the coordinate form while four take the generator form, among them the identity ciphertext helper, the placeholder written during an escrow refund and the two guards themselves.

Twisted Edwards points have a unique affine pair, and the two spellings do agree in the shipped runtime, so no defect follows today. What is missing is any statement of which form the type carries, whether the runtime keeps points normalised, or whether equality compares canonical representatives, and a reader cannot settle from the module that the two spellings denote one value. Were that property to fail, the guard could be satisfied by an identity key presented in whichever form the comparison did not recognise, and such a key removes the masking entirely, leaving the message point recoverable from an ElGamal ciphertext and the hybrid mask equal to a constant any observer can compute. That path is not reachable through this module, whose keys are derived inside the circuit at registration or read back from the registration map and never taken from a caller, so the guard stands here as protection for consumers that do accept a key as a parameter.

The same assumption has to survive across versions as well, because points are held in long-lived public state rather than converted to a fixed encoding first. The registration map stores each account's key as a curve point and both balance maps store ciphertexts whose components are themselves curve points, and every later operation reads those values back and compares or computes on them, the key check above included. A change to how the

runtime encodes or normalises a point would therefore have to preserve agreement with everything written under the present scheme, and no versioning or migration path exists for that state. This follows from the storage choice as it stands rather than from any predicted change.

Consider using one construction for the identity throughout, so that no reader has to determine whether two spellings denote the same value. Consider also documenting that equality on curve points compares canonical representatives, since two guards depend on it, and recording whether the runtime offers a stability guarantee for the encoding of persisted points, or storing a contract-controlled representation and checking supplied points against it where such a guarantee does not exist.

N-05 Incomplete Documentation For Native Token Supply Wrappers

Both supply wrappers, `NativeShieldedTokenFamilyPublicSupply` and `NativeShieldedTokenPublicSupply`, import and re-export the same two core maps. Ledger state is allocated once per module, so a contract composing both would receive a single pair. The single-token flavour keys its bucket to the unreserved all-zero `Bytes<32>`, while the per-domain flavour forwards an arbitrary caller domain. As a result, both wrappers would address the same entry, and the burn guard would stop binding per token type.

That composition is already prohibited where it originates. Both token flavours state that a contract must not import both for the same reason, and a supply wrapper is useful only alongside its token flavour. Every exported circuit of the two wrappers also shares a name with its counterpart at a different arity, so a dual import cannot be written unprefixed. The residual gap is documentary.

Consider recording in both supply wrappers that they share one set of core ledgers and that only one may be composed, matching the notice the token flavours carry. Deriving the single-token bucket from a fixed tag would not remove the condition, since the per-domain wrapper accepts any caller-supplied domain and no value can be reserved against it.

N-06 Lack Of Two-Step Ownership Transfer Modules

`ZOwnablePK` implements ownership by storing an `_ownerCommitment` and requiring privileged circuits to call `assertOnlyOwner`. This function recomputes an owner identifier

from `ownPublicKey()` and the witness `wit_secretNonce()`, and compares it against that commitment. A valid identifier is therefore a hash preimage, since `_computeOwnerId` returns `persistentHash` over the public key and the nonce.

Ownership transfer is a single step. `transferOwnership` accepts any `newOwnerId` other than the type default and commits it, with no step in which the recipient demonstrates that a preimage exists. For an arbitrary 32-byte value, no such preimage is findable, so once this value is committed, no caller can satisfy `assertOnlyOwner` and every owner-gated circuit becomes permanently unreachable. This includes the two circuits that might otherwise recover the situation, since a further `transferOwnership` and `renounceOwnership` both begin by calling `assertOnlyOwner`. Similarly, `Ownable.compact` too implements the single-step ownership transfer.

Consider implementing a two-step transfer module in which `transferOwnership` records a pending identifier and a separate acceptance circuit requires the recipient to prove knowledge of a public key and secret nonce reproducing it before `_ownerCommitment` is updated, similar to the Ownable and Ownable2Step contracts of the OpenZeppelin Solidity Library.

N-07 Missing Relation Between `ownPublicKey` and Its Associated Private Key

`assertOnlyOwner` derives an owner identifier from the bytes returned by `ownPublicKey` together with the witness `wit_secretNonce`, hashes the pair through `_computeOwnerId`, and compares the resulting commitment against the stored one. Nothing in that path establishes that the caller controls the private key belonging to the public key it supplies: there is no signature, no spend authorization, and no other in-circuit proof of key ownership.

The consequence is that the effective credential is the nonce alone. A public key contributes to the identifier but not to authentication, so an attacker who learns the nonce and presents the owner's public key bytes satisfies the gate and reaches every owner-restricted circuit, including `transferOwnership` and `renounceOwnership`. The module is consistent with itself here, its security considerations already treating nonce secrecy as load-bearing, so this is recorded as an informational observation rather than a defect. What is not stated is that the public key adds no security of its own, which a reader may not infer from a module named for public-key ownership, nor is any entropy requirement placed on the nonce that the credential now rests upon entirely.

Consider stating explicitly that the secret nonce is the sole authenticating secret, and specifying the entropy it must carry, so that an integrator does not treat the public key as a

second factor. Where a stronger guarantee is wanted, consider binding the check to a primitive that proves control of the key rather than merely quoting its bytes.

N-08 Owner Commitments Are Not Truly Distinct Over Multiple Contracts

Ownership in `ZOwnablePK` is represented by a public commitment, returned by `owner()` and derived by `_computeOwnerCommitment` as a `persistentHash` over the owner identifier, the `_instanceSalt`, a counter and a fixed domain tag. Of those four inputs only the salt varies between deployments, the tag being a constant, so the salt alone provides the per-instance namespace the module documents it as providing.

`initialize` rejects a zero owner identifier but persists the salt without validating it. Two deployments that share an owner identifier and are both initialized with the type default therefore publish identical commitments at the same counter value, and an observer can correlate administrative identity across those contracts by comparing a public getter. Authorization behavior is unchanged, since the commitment still requires a preimage to satisfy the owner check, so the consequence is the loss of the unlinkability the salt exists to provide.

Consider rejecting a salt equal to the type default in `initialize`, matching the check already applied to the owner identifier on the line above. Consider also folding a contract-unique value such as `kernel.self()` into the commitment, so that deployments remain namespaced even where a consumer supplies a poor salt.

N-09 Repeated Grants Can Exhaust Role Capacity And Permanently Block New Grants

Role grants are stored as leaves of `_operatorRoles`, a `MerkleTree<20, RoleCommitment>`, and the module header records that this gives a fixed capacity of 2^20 slots. Revocation does not reclaim a slot, since it records a nullifier in a separate set and leaves the original leaf in place.

What the header does not record is that a repeated grant of the same pairing consumes a further slot each time. The grant branch of `_updateRole` inserts the computed commitment unconditionally, with no check for a commitment that is already present, so issuing an identical role and account pairing many times exhausts capacity as surely as issuing distinct ones. Once the tree is full, no role can ever be granted again, and no path exists to recover a slot. Reaching that state requires the admin-gated grant circuit to be driven roughly a million times,

so the realistic route is automation repeating a grant it believes idempotent rather than an unprivileged caller. An attacker having access to the leaked admin key may become motivated to do so and cause denial-of-service on the contract.

Consider deduplicating before insertion, for instance by recording issued commitments in an auxiliary set and skipping an insert whose commitment is already present, which also makes a repeated grant idempotent as a consumer would expect. It is further advisable to expose the number of consumed slots, since the Merkle tree ledger type offers no occupancy accessor and a consumer therefore has no way to observe how much capacity remains.

N-10 Coin Information Is Recoverable Only From Discardable Return Values

Contract-initiated shielded sends create no coin ciphertext, so a recipient cannot discover a coin by scanning the chain, and the module records that the returned values are the only copies of those coins' information. Three circuits hand back such a value, a mint returning the recipient's only record, a burn returning the refund, and `_burnFromSelf` returning the change that replaces what the contract spent. Neither the absence of ciphertexts nor the decision to return the information is a defect, the former being a platform property and the latter the only mechanism available.

The defect is that nothing obliges a caller to retain what it receives. A circuit may be invoked for its effects with its result unused and the compiler accepts it, so a consumer treating one of these calls as void meets no error at compile time, no assertion at proving time, and no observable failure on chain. The obligation rests on prose, where a consumer is told it SHOULD persist the returned change. What is lost is not the value, which remains provably present, but every means of spending it. Burning from the contract's own holding is the sharpest case, the loss falling on the contract rather than on a user who chose to interact with it, and that holding never having been a ledger entry against which the omission could be reconciled.

Consider delivering the coin information through the contract rather than through the return value, by way of an optional extension that encrypts each created coin's fields to the recipient's public key and records the ciphertext in ledger state, so that a recipient discovers the coin by scanning contract state and decrypting with its own key. The primitive already exists as `EcdhMask`, which `ConfidentialFungibleToken` uses to deliver amounts through a per-account memo list. Two costs attach: recipients would need a registered encryption key, which this standard does not establish, and the stored ciphertexts would grow without bound. Both appear acceptable where the alternative is irrecoverable loss.

Alternatively, consider strengthening documentation from SHOULD to MUST and shipping a reference consumer.

N-11 Unbounded Per-Domain Supply Keys Can Create Persistent, Irreclaimable State Entries

`NativeShieldedTokenPublicSupplyCore` records its figures in two maps keyed by a 32-byte domain, and both update circuits write to them with a plain insert on whatever key reaches them, when recording an amount minted and when recording an amount burned. The key space is the full 32-byte range with no registration step or allowlist narrowing it, and neither circuit tests the amount against zero. A zero amount still reaches the insert, which writes the total back unchanged, and where the key has not been seen before that write adds an entry whose stored figure is zero, recording that a domain exists and nothing else. The two circuits differ in how easily that happens. Recording a burn requires the amount to fit within the domain's minted total, so on a key with no minted history only a zero amount gets through, whereas recording an amount minted checks only that the running total will not overflow before it writes and so accepts any amount on any key. Nothing in the module removes an entry once written, and the getters that answer a query through a membership test and a lookup therefore work against a structure that only grows.

Validation of the key rests entirely on the contract composing the extension. The per-domain wrapper passes its `domain` argument straight through and asserts nothing of its own, and the core's own guidance places the correctness of that argument with the consumer. A composition that exposes a supply path carrying a caller-chosen domain therefore lets each fresh value add state that neither the extension nor the consuming contract can reclaim, which raises the cost of every later supply read and write against a token whose accounting was expected to occupy one entry per token type.

Consider skipping the write when the amount is zero, so that a call which changes no total leaves no state behind, which on the burn side removes the only route to an entry under a domain that was never minted. Consider also confining the key space to domains the consuming contract registers or derives from a canonical token identifier, and recording in the wrapper documentation that a caller-chosen domain reaching these circuits produces persistent state with no route to reclaim it.

N-12 Role Renunciation Allows Unbounded Nullifier Growth

`ShieldedAccessControl` records granted roles as leaves of a Merkle tree and records revocations as entries in a nullifier set. Proving a role requires a membership path, which `_validateRole` checks before rejecting any commitment whose nullifier is present.

Revocation requires no such proof. `renounceRole` confirms only that the supplied account identifier matches the one derived from the caller's witness, then delegates to `_updateRole`, whose revoke branch inserts the nullifier without establishing that the role was ever granted. Both the role identifier and the account identifier are prover-chosen, the latter varying with the witness secret, so a caller can insert an unbounded number of nullifiers for pairings that never existed, each costing one transaction fee and persisting permanently. A second effect follows from the guard at the head of `_updateRole`, which rejects an already-revoked pairing on the grant branch as well: a nullifier inserted for a pairing blocks that pairing from ever being granted. That is confined to identities whose secret the caller holds, so it denies a role to the caller rather than to a third party.

Consider requiring a membership proof before a revocation is recorded, reusing the Merkle path `_validateRole` already checks, so that a nullifier can only exist for a role which was granted. It is further advisable to note in the module that a consumer with no need for self-revocation should not expose this circuit.

Conclusion

The Compact Contracts library provides reusable modules for building contracts on Midnight, covering fungible, non-fungible and multi-token standards, two confidential token models, access control, and the cryptographic and lifecycle utilities those depend on. OpenZeppelin audited the modules introduced in this release in full, and reviewed the pre-existing modules for the changes made to them.

The modules are small, single-purpose and well documented. Because Compact provides no inheritance, none is deployable on its own: the composing contract selects the entry points and supplies every access-control check, so a review of the library does not establish the properties of a deployment built on it. The same holds beyond the contract, since confidentiality depends on a wallet supplying fresh randomness and retaining the values circuits return, obligations no circuit can enforce.

The audit identified three high severity issues, four medium severity issues, and a number of low and note severity findings. The high severity issues concern an account identifier and an ElGamal secret scalar derived by the same hash with no domain separation, unread credits discarded by `clearMemos`, and a missing contract commitment claim when a mint or refund names a contract address as its recipient. The remainder are largely documentation and composition matters, arising not from defective logic but from obligations that a consuming contract must meet.

OpenZeppelin thanks the development team for their responsiveness throughout the engagement, and for resolving a number of platform-level questions that could not be settled from the contracts alone.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.